

Guide des commandes — odoo-module-migrator

Ce guide explique comment migrer des modules Odoo avec `odoo-module-migrate`, de l'installation à la lecture du rapport. La dernière partie présente les outils de mainteneur (régénérer les règles, bancs d'essai).

1. Installer

Il faut Python 3.11 ou plus récent et git.

```
git clone https://github.com/PSAI-Consulting/odoo-module-migrator.git
cd odoo-module-migrator
python -m venv .venv
.venv/Scripts/pip install -e .
```

Sous Linux / macOS, remplacez `.venv/Scripts/` par `.venv/bin/`.

Vérifier :

```
odoo-module-migrate --version
```

2. Migrer un module : les trois étapes

Étape 1 — Prévisualiser (rien n'est écrit)

```
odoo-module-migrate -d ./mes_modules -m mon_module -i 17.0 -t 20.0 --dry-run
```

L'outil migre une copie temporaire et affiche les différences (format `diff`). Vos fichiers ne sont pas modifiés.

Étape 2 — Migrer

Travaillez sur une **branche git** ou une **copie** du dossier : l'outil modifie les fichiers sur place.

```
odoo-module-migrate -d ./mes_modules -m mon_module -i 17.0 -t 20.0 --no-commit
```

Version recommandée, avec les sources de l'Odoo cible :

```
odoo-module-migrate -d ./mes_modules -m mon_module -i 17.0 -t 20.0 --no-commit \
  --odoo-root D:/Odoo/odoo/20.0 \
  --odoo-python D:/Odoo/venv20/Scripts/python.exe
```

Avec `--odoo-root` , l'outil :

- lance les **scripts officiels d'Odoo** (`odoo/upgrade_code` : `ir.access` , OWL 3, contraintes SQL...) ;
- vérifie que chaque **vue héritée** trouve ses ancres dans l'Odoo cible ;
- vérifie que les **modèles** hérités ou référencés existent encore ;
- vérifie les chemins `@api.depends` / `related=` et les imports `odoo.addons.*` .

`--odoo-python` est le Python capable d'importer l'Odoo cible (Odoo 20 : Python 3.12 ou plus). Les dossiers `enterprise/20.0` et `design-themes/20.0` voisins de `--odoo-root` sont trouvés automatiquement (sinon : `--addons-path`).

Étape 3 — Lire le rapport

Chaque module migré contient un fichier `MIGRATION_REPORT.md` :

Partie	Contenu
Résumé	niveau de risque, nombre de TODO, dont erreurs, transformations, fichiers modifiés
TODO manuels	par fichier, avec <code>fichier:ligne</code> cliquable :  erreur (bloque l'installation),  à vérifier
Transformations appliquées	ce que l'outil a réécrit
Fichiers modifiés	ajoutés, modifiés, supprimés

Traitez les  d'abord, puis installez le module sur une base de test.

3. Exemples courants

```
# Tous les modules d'un dossier
odoo-module-migrate -d ./mes_modules -i 16.0 -t 20.0 --no-commit

# Plusieurs modules précis
odoo-module-migrate -d ./mes_modules -m module_a,module_b -i 17.0 -t 18.0 --no-commit

# Rapports regroupés dans un dossier (un fichier par module + un sommaire)
odoo-module-migrate -d ./mes_modules -i 17.0 -t 20.0 --no-commit --report-dir ./rapports

# Les dépendances sont dans un autre dossier (lu seulement, jamais modifié)
odoo-module-migrate -d ./projet -m mon_module -i 17.0 -t 20.0 --no-commit \
    --odoo-root D:/Odoo/odoo/20.0 --odoo-python D:/Odoo/venv20/Scripts/python.exe \
    --context-path ./autres_modules

# Ignorer les modules OCA présents dans le dossier
odoo-module-migrate -d ./mes_modules -i 17.0 -t 20.0 --no-commit --no-oca-modules

# Journal complet dans un fichier
odoo-module-migrate -d ./mes_modules -i 17.0 -t 20.0 --no-commit --log-path migration.log
```

Relancer la migration sur un module déjà migré ne change rien (l'outil est idempotent).

4. Toutes les options

Indispensables

Option	Rôle
<code>-i</code> , <code>--init-version-name</code>	Version de départ : 8.0 à 19.0 (obligatoire)
<code>-t</code> , <code>--target-version-name</code>	Version cible : 9.0 à 20.0 (défaut : la plus récente)
<code>-d</code> , <code>--directory</code>	Dossier contenant les modules (défaut : dossier courant)
<code>-m</code> , <code>--modules</code>	Modules à migrer, séparés par des virgules (défaut : tous)

Sécurité et contrôle

Option	Rôle
<code>-n</code> , <code>--dry-run</code>	Prévisualiser : affiche les différences, n'écrit rien
<code>-nc</code> , <code>--no-commit</code>	Ne pas créer de commit git (recommandé : vous relisez avant de commiter)
<code>-npc</code> , <code>--no-pre-commit</code>	Ne pas lancer pre-commit après la migration
<code>-nrmf</code> , <code>--no-remove-migration-folder</code>	Garder les dossiers <code>migrations/</code> des modules (supprimés par défaut)
<code>--set-installable</code>	Forcer <code>'installable': True</code> (par défaut, un module désactivé exprès reste désactivé et c'est signalé)

Odoo cible (recommandé à partir de 18.0)

Option	Rôle
<code>--odoo-root</code>	Sources de l'Odoo cible (ex. <code>D:/Odoo/odoo/20.0</code>)
<code>--odoo-python</code>	Python capable d'importer cet Odoo
<code>--addons-path</code>	Addons de référence, séparés par des virgules (défaut : <code>enterprise</code> et <code>design-themes</code> voisins)
<code>--context-path</code>	Autres dossiers de modules où chercher les dépendances (lecture seule)

Rapport et journal

Option	Rôle
<code>--report-dir</code> DOSSIER	Écrire les rapports dans ce dossier (<code><module>.md</code> + <code>README.md</code> récapitulatif) au lieu de <code>MIGRATION_REPORT.md</code> dans chaque module
<code>--no-report</code>	Pas de rapport
<code>-ll</code> , <code>--log-level</code>	<code>DEBUG</code> , <code>INFO</code> (défaut), <code>WARNING</code> , <code>ERROR</code> , <code>CRITICAL</code>
<code>-lp</code> , <code>--log-path</code> FICHER	Écrire le journal dans un fichier
<code>-lpwo</code> , <code>--log-path-warninglevelonly</code> FICHER	Écrire seulement les avertissements et erreurs (Markdown)

Divers

Option	Rôle
<code>--no-oca-modules</code>	Ne pas migrer les modules OCA trouvés dans le dossier
<code>--oca-file-list</code>	Écrire <code>OCA_MODULES.md</code> : liste des modules OCA du projet
<code>-fp</code> , <code>--format-patch</code>	Récupérer le module depuis la branche de la version précédente du dépôt (<code>git format-patch</code> , un seul module)
<code>-rn</code> , <code>--remote-name</code>	Remote git utilisé par <code>--format-patch</code> (défaut : <code>origin</code>)
<code>--version</code>	Afficher la version de l'outil
<code>-h</code> , <code>--help</code>	Afficher l'aide

5. Questions fréquentes

Le module ne s'installe toujours pas après la migration. C'est normal pour une partie des modules : certaines évolutions d'Odoo ne peuvent pas être réécrites automatiquement (une vue cible a disparu, un modèle a été supprimé...). Elles sont toutes dans le rapport, avec `fichier:ligne` . Commencez par les  .

Un TODO me paraît faux. Vérifiez dans les sources de l'Odoo cible. Si le TODO est vraiment faux, c'est un bug de l'outil : signalez-le avec l'extrait de code concerné.

Puis-je migrer de 12.0 directement en 20.0 ? Oui : l'outil enchaîne tous les sauts. Le rapport regroupe les TODO de tous les sauts.

Mes fichiers changent-ils d'encodage ou de fins de ligne ? Non : l'encodage, le BOM et les fins de ligne (CRLF / LF) de chaque fichier sont conservés.

6. Outils de mainteneur

Ces outils servent à faire évoluer l'outil lui-même. Ils sont dans `tools/` et ne sont pas installés avec la commande.

Régénérer les règles depuis les sources Odoo

Une seule commande, `tools/extract/extract_changes.py` , avec une sous-commande par type de règle. Elle lit des dépôts git **bare** d'Odoo (aucun checkout) :

```
# Modules supprimés / renommés / fusionnés
python tools/extract/extract_changes.py modules --from 19.0 --to 20.0 \
    --repo D:/Odoo/.repos/odoo.git@addons --repo D:/Odoo/.repos/enterprise.git \
    --openupgrade D:/Odoo/local-addons/OdooOCA/OpenUpgrade \
    --output
odoo_module_migrate/migration_scripts/deprecated_modules/migrate_190_200/modules.yaml
```

Sous-commande	Produit
modules	modules supprimés, renommés, fusionnés
fields	champs renommés / supprimés (OpenUpgrade + sources, <code>--sources</code>)
models	modèles supprimés / renommés
api	API Python retirée (candidats à relire)
js	modules JS supprimés ou déplacés
views	vues supprimées (héritages cassés)
model-modules	module qui définit chaque modèle
tools-exports	noms que <code>from odoo.tools import X</code> ne fournit plus

Détail des options : `python tools/extract/extract_changes.py <sous-commande> --help` . Les fichiers produits s'appellent `generated.yaml` ; les corrections faites à la main vont dans `curated.yaml` (voir [FONCTIONNEMENT.md](#)).

Bancs d'essai : migrer puis installer sur une base Odoo vierge

Un banc est décrit par un fichier YAML (`tools/bench/benches/*.yaml`) : versions, sources Odoo, modules à tester.

```
# Banc complet : export, migration, contrôles, installation, résultats
python tools/bench/bench.py tools/bench/benches/example_oca_17_20.yaml --work D:/tmp/bancs

# Boucle rapide : refaire seulement les modules en échec
python tools/bench/bench.py tools/bench/benches/example_oca_17_20.yaml --work D:/tmp/bancs --
retry-failed

# Migration et contrôles seulement, sans base de données
python tools/bench/bench.py tools/bench/benches/example_oca_17_20.yaml --work D:/tmp/bancs --
skip-install
```

Résultat : `<work>/<banc>/RESULTS.md` (un module par ligne : installé ou non, première erreur, lien vers son rapport).

Les bases créées s'appellent toujours `migrator_bench_*` et sont supprimées à la fin ; aucune autre base n'est touchée.

Outil	Rôle
<code>bench/bench.py</code>	banc complet ou boucle rapide (<code>--retry-failed</code>)
<code>bench/bench_install.py</code>	installation par lots sur une copie de base vierge (utilisé par le banc)
<code>bench/check_modules.py</code>	contrôles statiques d'un dossier de modules
<code>bench/ground_truth.py</code>	compare la migration de l'outil à celle faite à la main par l'OCA